Web-Technologien UE - Assignment 4

Raphael Bleier, WS 2026

Dateien: In `ex1` gibt es zu jeder Teilaufgabe eine eigene JS-Datei mit passender HTML (`productsA_b1.html` usw.). `shopA_final.js` ist alles zusammen: Arrow Function, Empty Cart und Sortierung. Ex3 a/b/c/e, Ex4 und Ex5 laufen mit `node datei.js`.

Exercise 1 - Closures (shopA.js)

Code erklärt

`'use strict'` schaltet den Strict Mode ein, damit werden z.B. nicht deklarierte Variablen zum Fehler. `products` ist ein globales Array, in dem alle Produkte als Objekte landen.

`updateCart()` leert die Liste `#orderedItems`, geht dann alle Produkte durch und erzeugt für jedes mit `numInCart > 0` ein `li` mit Titel, Preis und Anzahl. Nebenbei wird der Gesamtpreis aufsummiert und mit zwei Kommastellen in `#totalPrice` geschrieben.

Die for-of Schleife geht über alle `div.product`. Für jedes div wird ein neues Objekt `cp` gebaut. Die Daten kommen aus dem HTML: die id aus `data-prodId` (im dataset wird das zu `prodid`, weil HTML-Attribute klein geschrieben werden), der Titel über `firstChild.nextSibling.firstChild.nodeValue`. firstChild ist der Whitespace-Text, nextSibling davon das h2 und dessen firstChild der Text. Beschreibung und Preis kommen über die Klassen. `parseFloat("2890,00")` liest nur bis zum Komma, also 2890. Dazu kommen `numInCart = 0` und eine Referenz aufs div. Das Objekt kommt ins Array.

Dann wird in jedem Schleifendurchlauf eine Funktion `addListener` definiert, die `cp.numInCart` um 1 erhöht und den Warenkorb neu zeichnet. Sie wird als click-Listener auf das `.buy` div gehängt.

a.a) Warum kennt addListener das richtige cp?

Weil das eine Closure ist. addListener wird innerhalb vom Schleifenblock definiert und merkt sich die Umgebung, in der sie entstanden ist, also auch die Variable cp aus genau diesem Durchlauf. Wenn später geklickt wird, ist die Schleife längst fertig, aber die Funktion hat ihre Referenz auf "ihr" cp noch. Da cp mit `let` deklariert ist, gibt es pro Durchlauf eine eigene cp-Variable.

a.b) var statt let

Dann zählt jeder Button immer das letzte Produkt hoch (Tamron). Getestet in `productsA_a_var.html`: Einmal Sony Alpha und einmal Sony Lens geklickt ergibt "Tamron 17-24mm 800(2)".

Grund sind die Scopes. `let` und `const` sind block-scoped, sie gelten nur im Block `{ }` und bei Schleifen wird pro Durchlauf eine neue Variable angelegt. `var` ist function-scoped bzw. hier global. Es gibt nur eine einzige cp-Variable für die ganze Schleife, die bei jedem Durchlauf überschrieben wird. Alle vier Closures zeigen auf dieselbe Variable, und am Ende steht da das letzte Produkt drin. Außerdem wird var gehoisted, man könnte cp schon vor der Schleife verwenden (wäre dann undefined).

b) Anonyme Funktion / Arrow Function

Siehe `shopA_b1.js` (`function (e) {...}`) und `shopA_b2.js` (`(e) => {...}`). Funktioniert beides gleich, weil im Handler kein `this` verwendet wird. Die Closure über cp bleibt dieselbe.

c) Empty Shopping Cart

`shopA_c.js`: Click-Listener auf `#emptyCard`, der bei allen Produkten numInCart auf 0 setzt und updateCart aufruft.

d) Sortieren nach Preis

`shopA_d.js`: Eine Kopie vom products Array wird mit `sort((a, b) => a.price - b.price)` sortiert. Dann wird jedes `domElement` in dieser Reihenfolge vor das "Your Order" div gehängt. Weil ein DOM-Element nur einmal existieren kann, wird es dabei verschoben und nicht kopiert. Ergebnis: Tamron (800) < Nicon (900) < Sony Lens (1300) < Sony Alpha (2890).

Exercise 2 - Objekte und this (shopB.js)

a) Code erklärt

`productList` ist ein Objekt, das dasselbe products Array referenziert. `listProducts()` loggt für jedes Produkt im Warenkorb den Titel vom nächsten Produkt in der Liste (mit Modulo, damit nach dem letzten wieder das erste kommt). `updateCart()` ist wie bei shopA, ruft aber `p.render()` für den Text auf und am Ende `listProducts()`.

`Product(el)` ist eine Konstruktorfunktion. Mit `new Product(pdiv)` wird ein neues Objekt erzeugt, this zeigt darauf. Es liest die gleichen Daten wie vorher aus dem HTML, bekommt die Methoden updateProduct und render und hängt updateProduct als click-Listener an den buy Button.

a.a) Warum geht der Warenkorb nicht?

Problem 1: `clickButton.addEventListener('click', this.updateProduct)` übergibt nur die Funktion, nicht das Objekt dazu. Wenn der Browser den Handler aufruft, ist this das Element, an dem der Listener hängt, also das `.buy` div. `this.numInCart += 1` schreibt dann NaN ins div und das Produkt bleibt bei 0. Fix: `() => this.updateProduct()`. Die Arrow Function hat kein eigenes this und nimmt das vom Konstruktor, also das Produkt. `this.updateProduct.bind(this)` würde auch gehen.

Problem 2: In `listProducts` wird an `forEach` eine normale `function` übergeben. Darin ist this im Strict Mode `undefined`, also kommt sobald ein Produkt im Warenkorb ist "Cannot read properties of undefined (reading 'products')". Fix: auch hier eine Arrow Function, dann ist this das productList Objekt.

Gefixte Version in `ex2/shopB.js`. Mit Sony Alpha und Tamron im Warenkorb loggt sie "Sony 24-105mm lens;" und "Sony Alpha IV;", also jeweils das nächste Produkt.

a.b) Wie oft ist render im Speicher?

4 mal. `this.render = function () {...}` steht im Konstruktor, also wird bei jedem `new Product` ein neues Funktionsobjekt erzeugt. Bei 4 Produkten gibt es 4 Kopien (genauso bei updateProduct). Über den Prototype wäre es nur einmal da.

b) productsC.js mit Prototype

Alle Methoden hängen jetzt an `Product.prototype` (updateProduct, render, removeOneFromCart, getTotalPrice) und sind nur einmal im Speicher. Im Konstruktor stehen nur noch die Daten und die Event Listener.

a) Der "remove one" Button wird per JS erzeugt und in ein eigenes div neben das buy div gehängt. Zuerst hab ich ihn ins buy div gesteckt, aber dann bubbelt der Klick zum Listener vom buy div und es wird gleich wieder 1 dazugezählt. updateCart setzt `disabled` je nachdem ob numInCart größer 0 ist.

b) `getTotalPrice()` gibt price · numInCart zurück und wird in der Warenkorb-Zeile angezeigt, z.B. "Sony 24-105mm lens 1300(1) = 1300.00".

c) Nur products[0] bekommt eine eigene `render` Property. Weil JS beim Suchen zuerst am Objekt selbst schaut und erst dann im Prototype, überdeckt die das prototype-render nur für dieses eine Objekt. Darin rufe ich mit `Product.prototype.render.call(this)` die normale Version auf und hänge " New Arrival!" an. Im HTML heißt das erste Produkt übrigens Sony Alpha IV, nicht A7V.

Exercise 3 - Closures und Currying

a) Higher Order Functions

Eine Higher Order Function ist eine Funktion, die eine andere Funktion als Parameter nimmt oder eine Funktion zurückgibt (oder beides). Das geht, weil Funktionen in JS First Class Functions sind: Sie sind ganz normale Werte, man kann sie Variablen zuweisen, übergeben, zurückgeben und in Arrays oder Objekten speichern.

Mein Beispiel in `a_higher_order.js`: `applyTwice(fn, value)` ruft fn zweimal auf, `applyTwice(double, 5)` gibt 20. `makeGreeting("Hallo")` gibt eine Funktion zurück.

Drei Beispiele aus JS selbst: `Array.map/filter/reduce` (Callback pro Element), `addEventListener` (Handler wird übergeben) und `setTimeout/setInterval`.

b) Closure

Eine Closure ist eine Funktion zusammen mit der Umgebung, in der sie definiert wurde. Sie kann auf die Variablen der äußeren Funktion zugreifen, auch nachdem diese schon fertig ist. In `b_closure.js` gibt `makeCounter()` eine Funktion zurück, die eine lokale Variable count hochzählt. counterA gibt 1, 2, 3 aus, counterB fängt wieder bei 1 an, weil jeder Aufruf von makeCounter eine eigene count-Variable bekommt. Von außen kommt man an count nicht ran, das ist quasi eine private Variable.

c) Currying

Currying heißt, eine Funktion mit mehreren Parametern so umzubauen, dass sie immer nur einen Parameter nimmt und eine Funktion für den nächsten zurückgibt. Aus `price(net, tax, discount)` wird `curriedPrice(net)(tax)(discount)`. Praktisch, weil man Parameter vorbelegen kann. In `c_currying.js` mach ich mir damit eine Funktion, die schon die österreichische Steuer drin hat. Kurzform: `const add = a => b => c => a + b + c;`

d) getEventHandler

Siehe `ex3/shopB_d.js`:

```
this.getEventHandler = function () {
    return () => this.updateProduct();
}
clickButton.addEventListener('click', this.getEventHandler());
```

Die Arrow Function ist wichtig, weil sie kein eigenes this hat. Sie nimmt das this von getEventHandler, und das ist beim Aufruf `this.getEventHandler()` das Produkt. Mit einer normalen `function` wäre this beim Klick wieder das div und wir hätten den gleichen Fehler wie in Ex2.

e) Was wird ausgegeben?

```
20
21
21
16
```

`myFunct(12, 8)`: a = 12 ist kleiner als 20, also wird `calc(12)` direkt aufgerufen. Das gibt 12 + 8 = **20** aus, x ist 20.

`myFunct(21, 7)`: a = 21, also else-Zweig. Es wird **21** ausgegeben und die Funktion calc selbst zurückgegeben (nicht aufgerufen). calc ist eine Closure und merkt sich b = 7.

`y(14)`: calc(14), also 14 + 7 = **21**.

`y(9, 33)`: calc hat nur einen Parameter, die 33 wird einfach ignoriert. 9 + 7 = **16**.

Exercise 4 - Prototypes

Alles in `ex4/ex4.js`.

a) Objekt-Literal mit width, height, bitdepth und computeSize(). pixels = width · height, rawsize = pixels · bitdepth / 8 (bits in bytes). Für 1920x1080 mit 24 Bit: 2.073.600 Pixel, 6.220.800 Bytes.

b) Konstruktor `Image`, rawsize und pixels werden direkt im Konstruktor berechnet. print() geht mit `for...in` über alle Properties. Funktionen lasse ich dabei weg, sonst würde der Quellcode von print mit ausgegeben.

c) `Image.prototype.printMore` ruft print auf und gibt dann landscape, portrait oder square aus.

d) print vs. printMore

print wird im Konstruktor mit `this.print = ...` angelegt. Jedes Objekt hat also seine eigene Kopie als eigene Property. printMore gibt es nur einmal im Prototype und alle Image-Objekte teilen sich die über die Prototype-Kette.

Ja, man kann sie zur Laufzeit umbiegen, sind ja nur Properties. `img3.print = function () {...}` ändert print nur für img3, die anderen Bilder behalten ihre Version. Weil printMore intern `this.print()` aufruft, verwendet `img3.printMore()` danach automatisch die neue print. Ändert man `Image.prototype.printMore`, ändert sich das für alle Objekte auf einmal, auch für schon erzeugte. Aufpassen muss man bei `img3.print = img3.printMore`, das gibt eine Endlosschleife, weil printMore wieder print aufruft.

e) Video

`Video` ruft mit `Image.call(this, width, height, bitdepth)` den Image-Konstruktor auf, damit die Image-Properties und print auch im Video landen. Dazu kommen duration und framerate. Mit `Video.prototype = Object.create(Image.prototype)` wird die Prototype-Kette verbunden. totalFrames() gibt duration · framerate zurück: 60 s mit 30 fps ergeben 1800 Frames, 15 s mit 60 fps ergeben 900.

f) printMore auf Video?

Ja, weil Video.prototype von Image.prototype erbt und JS printMore über die Kette findet (`video2 instanceof Image` ist true). Ohne die Zeile mit Object.create würde nur `Image.call` laufen. Dann hätte das Video zwar print (das wird ja im Konstruktor gesetzt), aber kein printMore, und es käme "printMore is not a function".

Exercise 5 - Klassen

Siehe `ex5/ex5.js`, gibt das gleiche aus wie Ex4. `class Image` mit constructor, print() und printMore(), dazu `class Video extends Image` mit `super(...)` und totalFrames(). Unterschied zu vorher: Methoden in einer Klasse landen automatisch im Prototype, also ist hier auch print nur einmal im Speicher. Klassen sind in JS eigentlich nur eine schönere Schreibweise für Prototypes. `extends` macht intern das gleiche wie Image.call plus Object.create. Zuweisen zur Laufzeit funktioniert genauso.

Konsolenausgabe ex4.js (gekürzt)

```
a) pixels: 2073600 rawsize: 6220800 bytes
c) printMore:  ... -> landscape / -> portrait / -> square
d) kurz: 800x800, 2560000 bytes
   -> square
e) ... duration: 60  framerate: 30  totalFrames: 1800
   ... duration: 15  framerate: 60  totalFrames: 900
f) ... -> portrait
   video2 instanceof Image: true
```